



UNIVERSITY OF HELSINKI
FACULTY OF SCIENCE

Programming Projects in Molecular Modelling
Prof. Alex Bunker

Project 4

Thermostats and Monte Carlo Simulation

Md. Shafayet Islam
Luca Mössinger

March 2025

Contents

1. Introduction	3
2. Methods & Implementation	4
2.1. Equilibration and Scaling	4
2.2. Behrendsen and Langevin Thermostats	4
2.3. Monte-Carlo Simulation	6
2.3.1. System Initialization	6
2.3.2. Potential Energy Evaluation	6
2.3.3. Monte Carlo moves and Metropolis Algorithm	6
2.3.4. Simulation Protocol	7
3. Result and Analysis	8
3.1. Equilibration Temperature	8
3.2. Behrendsen and Langevin Thermostats	9
3.3. Temperature and Density Effect in 2D LJ simulation	18
3.4. Monte Carlo Simulation	21
3.5. Effect of Density, Temperature, and Phase	22
3.5.1. Temperature effect	23
3.6. Computational efficiency and performance Optimization	25
3.7. Cython Optimization	26
4. Conclusion	26
Bibliography	27
5. Appendix	28
A. Graphs of MC simulation. Varied temperature	28
B. Tables	29
C. List of MD simulations core functions and Description	29
D. List of MC simulations core functions and Description	32

1. Introduction

Previously, we explored MD simulations for particles in a box. We applied the leapfrog and linked-cell algorithms to run an MD simulation. In this assignment, we are pushing ourselves a little bit further.

In this assignment, we explore two-dimensional Lennard-Jones systems' thermodynamic and structural properties through molecular dynamics and Monte Carlo simulations. We first examine system equilibration in NVE ensembles, analyzing energy convergence and temperature fluctuations as the system approaches steady-state behavior. We implemented both the Berendsen and Langevin thermostats in our code. After that, we evaluate thermostat performance by comparing Langevin and Berendsen's methods for maintaining target temperatures while preserving proper fluctuation statistics. We also characterize phase behavior through structural analysis and radial distribution functions across varying densities and temperatures, identifying distinct regimes of particle organization. Finally, we implemented a Monte Carlo(MC) simulation on the 2D particle. The study contrasts the sampling efficiency of molecular dynamics and Monte Carlo approaches, particularly focusing on displacement optimization in MC simulations and time-step stability in MD simulations. We also show the efficiency and performance of this code.

We use `python3` for coding all our assignments. Finally, as Python seems to be slow, we also try to improve the performance of the code using `cython`. The code documentation of the program structure is in the appendix section. All the output trajectories and final output coordinates are in the XYZ file format. The VMD application performed all the visualizations, and the plots were made using a simple Python script.

2. Methods & Implementation

2.1. Equilibration and Scaling

In this part of the project, we build on Project 3, where the linked-cell algorithm (LCA) was implemented to efficiently compute forces in molecular dynamics simulations with short-range interactions such as the Lennard-Jones potential.[2] The key improvement of the LCA is its ability to reduce computational complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$, enabling us to simulate larger systems with improved performance while maintaining physical accuracy.[1]

With a working LCA-based code in place, we focused on analyzing the potential and kinetic energy as a function of time during an NVE simulation. The system is initialized in a non-equilibrium state (e.g., with a lattice and random velocity noise), and evolves under conservative forces. The evolution of energy components reveals large fluctuations and oscillations in both potential and kinetic energy initially. These are indicative of the system undergoing an equilibration process, where particles rearrange and redistribute energy due to interactions. The total energy remains nearly constant, as expected in a well-behaved NVE system. Over time, the potential energy decreases slightly while the kinetic energy increases, indicating a relaxation toward a more mechanically stable configuration.

Using the kinetic energy, we compute an instantaneous temperature-like quantity via the equipartition theorem:

$$T(t) = \frac{2K(t)}{N_{\text{dof}}}$$

where $K(t)$ is the kinetic energy at time t , and $N_{\text{dof}} = N \cdot d$ is the number of degrees of freedom.[1]

2.2. Behrendsen and Langevin Thermostats

We extend the molecular dynamics framework to include temperature control via thermostats, allowing us to study how different methods influence temperature evolution and fluctuations in a system of particles interacting through the Lennard-Jones potential.

We incorporate two thermostats: Behrendsen and Langevin. Both are applied after each velocity update step to regulate the system's temperature, but they operate based on different principles.

The Behrendsen thermostat adjusts the particle velocities by scaling them toward the

2. Methods & Implementation

target temperature. This is achieved through a relaxation mechanism:

$$\vec{v}_i \rightarrow \lambda \vec{v}_i \quad \text{with} \quad \lambda = \sqrt{1 + \frac{\Delta t}{\tau} \left(\frac{T_0}{T} - 1 \right)}$$

where T_0 is the target temperature, T is the instantaneous temperature, τ is a relaxation time, and Δt is the integration time step. This method rapidly brings the system toward the desired temperature but does not generate realistic thermal fluctuations.

The Langevin thermostat instead mimics a heat bath by adding friction and stochastic noise to particle motion:

$$\vec{v}_i \rightarrow c_1 \vec{v}_i + c_2 \vec{\eta}_i$$

where $c_1 = e^{-\gamma \Delta t}$, $c_2 = \sqrt{(1 - c_1^2) T_0}$, and $\vec{\eta}_i$ is a Gaussian random vector. The Langevin approach maintains a steady average temperature while allowing for realistic fluctuations.[1]

The simulation starts by placing particles in a lattice with small random displacements, and assigning velocities drawn from a Maxwell-Boltzmann distribution corresponding to the target temperature. As the system evolves, we compute forces using the LCA, integrate positions and velocities using the velocity-Verlet scheme, and apply the selected thermostat after each step. Periodic boundary conditions ensure spatial continuity and avoid surface effects.

At each time step, we compute the instantaneous temperature via the kinetic energy:

$$T(t) = \frac{2K(t)}{N_{\text{dof}}}$$

where $K(t)$ is the total kinetic energy and $N_{\text{dof}} = N \cdot d$ is the number of degrees of freedom.[1]

Furthermore, the effect of varying the simulation timestep Δt is analyzed to determine the maximum timestep for which the algorithm remains numerically stable. This behavior is studied systematically in both two- and three-dimensional systems.

We also explore how varying the thermostat relaxation parameter τ (for Behrendsen) influences temperature regulation and system dynamics. This sensitivity analysis reveals trade-offs between temperature control accuracy and numerical stability.

To probe transport properties, we measure the diffusion coefficient D as a function of both real time and simulation time across the two thermostats and the NVE ensemble. This is complemented by an analysis of the resulting temperature distributions for each method, highlighting the distinct behavior of thermostats with respect to thermal fluctuations and equilibrium characteristics.

A flow diagram of how the code works is given in the Appendix 17

2. Methods & Implementation

2.3. Monte-Carlo Simulation

2.3.1. System Initialization

A total of N particles were placed in a square box of length L , representing a constant area system under periodic boundary conditions. The initial configuration was generated using a triangular lattice arrangement to promote an ordered start. If the lattice could not accommodate all particles, the remaining ones were placed randomly within the box. This ensured an approximately uniform initial density, calculated as $\rho = \frac{N}{L^2}$. The initial potential energy of the system was then computed using the Lennard-Jones (LJ) potential, incorporating a cutoff radius r_c to ignore negligible long-range interactions. The system initialization is done in the `initialize_lattice()` function.

2.3.2. Potential Energy Evaluation

The interparticle interactions were modeled using the standard Lennard-Jones potential:

$$V(r) = \begin{cases} 4 \left[\left(\frac{1}{r} \right)^{12} - \left(\frac{1}{r} \right)^6 \right], & \text{if } r < r_c \\ 0, & \text{otherwise} \end{cases}$$

The energy between each pair of particles was computed using the minimum image convention, accounting for periodic boundary conditions. This was efficiently implemented in the `periodic_distance()` function, which calculated the shortest distance between two particles across the periodic box.

2.3.3. Monte Carlo moves and Metropolis Algorithm

We employed a Monte Carlo (MC) simulation using the Metropolis algorithm to study a system of interacting particles confined in a two-dimensional periodic box. The MC sampling involved the random selection of a single particle and proposing a displacement in a random direction within a circle of maximum radius. The proposed displacement was generated in polar coordinates, ensuring uniform sampling within a circular area. The new position was wrapped using periodic boundary conditions. The energy difference ΔE between the new and old configurations was calculated by evaluating pairwise energy contributions of the moved particle. The move was then accepted or rejected based on the Metropolis criterion:

$$P_{\text{accept}} = \min \left(1, e^{-\beta \Delta E} \right)$$

where $\beta = 1/T$ is the inverse temperature. To prevent numerical overflow, the exponent was clipped to a maximum value. The whole simulation is run by the `run_simulation()` function.

2. *Methods & Implementation*

2.3.4. Simulation Protocol

The simulation was run in two phases:

1. **Equilibration Phase/Burin-Phase:** A set number of `equil_steps` were performed to allow the system to reach a thermodynamically stable state. The energy was updated iteratively, but no data were collected during this phase.
2. **Production Phase:** After equilibration, `steps` MC cycles were executed. During this phase, the energy and other observables were sampled at regular intervals specified by `sample_freq`. Accepted moves were tracked to compute the acceptance ratio, which served as an indicator of simulation efficiency.

A flow diagram of how the functions/engine works together is given in the Appendix 18

3. Result and Analysis

3.1. Equilibration Temperature

The calculation of the equilibration temperature is based on the method described in section 2.1 and was performed for 1000 particles. Figure 1 shows the evolution of the temperature during minimization.

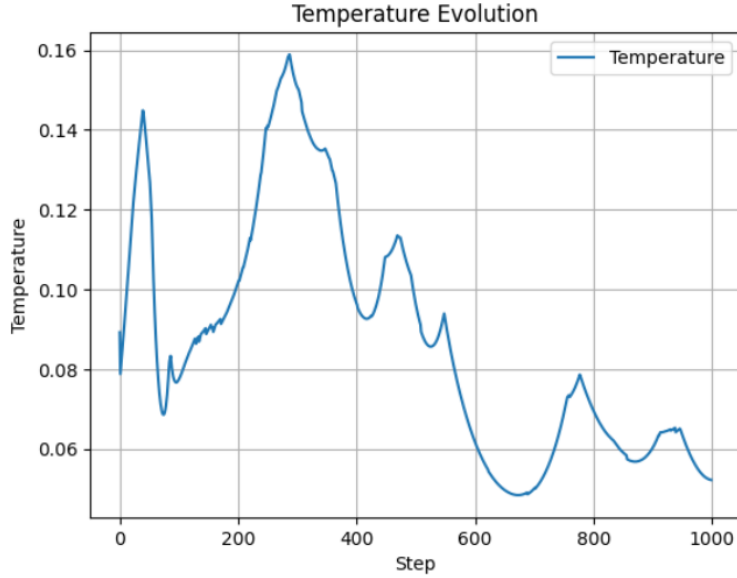


Figure 1: Temperature evolution during energy minimization. The system starts in a high-energy, non-equilibrium configuration and evolves toward a lower-energy state. The temperature exhibits damped oscillations due to residual kinetic motion and force relaxation. As minimization proceeds, the oscillations decrease in amplitude, and the system gradually approaches a mechanically stable configuration with low residual temperature.

In the plot (steps 0–200), the temperature fluctuates and even increases, which may seem counterintuitive for a minimization. The system enters a regime of repeated climbing and relaxing, visible as peaks in the temperature around steps 300–500. This reflects transitions between local minima in the potential energy surface. Beyond step 500, the temperature steadily decreases, with smaller oscillations. The system is approaching a stable configuration. Eventually, temperature approaches zero, indicating the system is near a mechanical equilibrium.

In this post-equilibration regime, we sample the temperature over time and construct a histogram of its fluctuations (see figure 2).

3. Result and Analysis

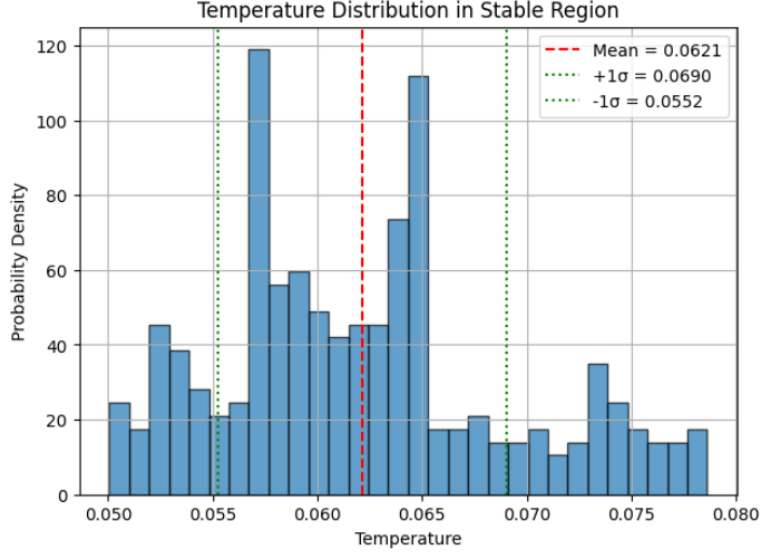


Figure 2: Temperature distribution in the stable region of energy minimization. The histogram shows the distribution of instantaneous temperatures measured during the final 300 steps of the minimization. The system exhibits a mean temperature of 0.0621 with relatively narrow fluctuations, as indicated by the standard deviation bounds ($\pm 1\sigma$). The presence of multiple peaks reflects subtle oscillatory modes or local rearrangements persisting in the minimized configuration, but the overall spread remains low, suggesting mechanical stabilization has been largely achieved.

As system size grows the mean temperature remains constant, as energy per particle is conserved. The relative fluctuations in temperature decrease, consistent with statistical mechanics: larger systems are more stable thermodynamically, and quantities like temperature become more sharply defined.

3.2. Berendsen and Langevin Thermostats

In this section, we present the analysis of the temperature control and numerical stability of the implemented thermostats (see 2.2), highlighting their performance and physical accuracy under various simulation conditions.

To evaluate thermostat performance, we monitor both the time evolution and fluctuations of the system's temperature. Figure 3 shows that the Berendsen thermostat rapidly drives the system toward the target temperature, effectively damping deviations. The target temperature of 0.1 was chosen as a standard low-temperature reference to ensure stability and comparability across simulations, even though the system naturally settles at a slightly lower value during minimization. The Langevin thermostat, while also

3. Result and Analysis

converging, does so more gradually and with persistent fluctuations over time.

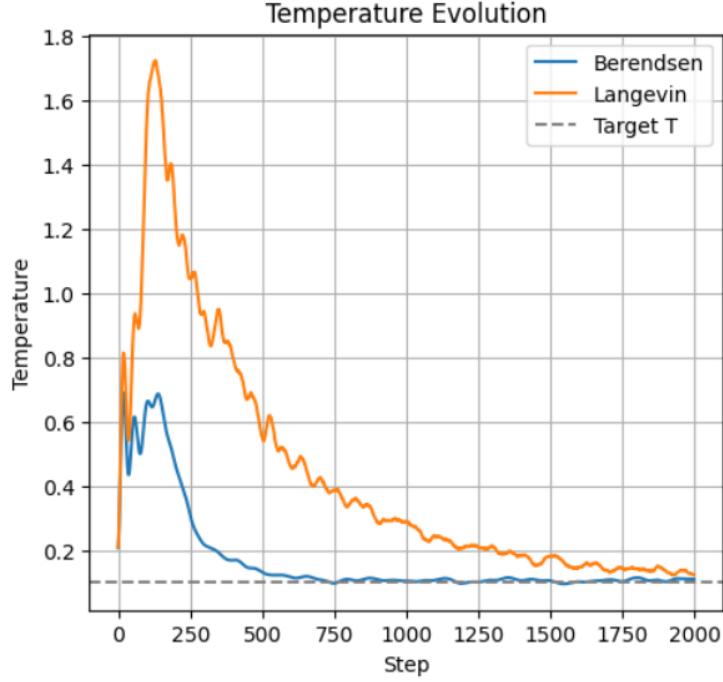


Figure 3: Temperature evolution under Berendsen and Langevin thermostats. Both methods drive the system toward the target temperature of 0.1, with the Berendsen thermostat showing rapid damping and suppressed fluctuations, while the Langevin thermostat exhibits a slower, noise-driven relaxation consistent with canonical ensemble behavior.

Figure 4 highlights a crucial difference in temperature fluctuation behavior. The Berendsen thermostat produces a very narrow distribution centered tightly around the target temperature. This indicates that fluctuations are heavily suppressed due to its velocity rescaling nature. In contrast, the Langevin thermostat leads to a broader and more irregular temperature distribution, with a mean temperature noticeably above the target. This reflects the influence of stochastic noise and friction in Langevin dynamic.

3. Result and Analysis

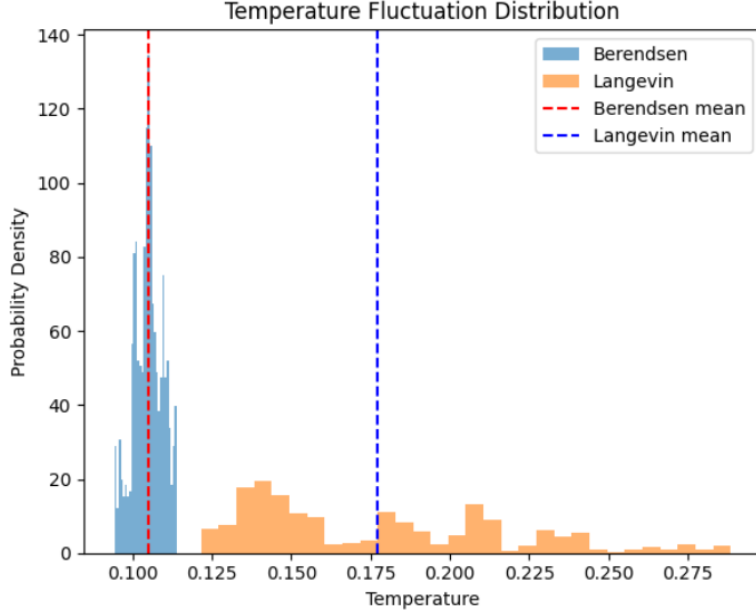


Figure 4: Temperature fluctuation distributions for Berendsen and Langevin thermostats. The Berendsen ensemble produces a narrow, sharply peaked distribution centered near the target temperature, reflecting its strong suppression of thermal fluctuations. In contrast, the Langevin ensemble exhibits a broader distribution with a higher mean temperature, indicating larger stochastic fluctuations around the target due to its physically motivated noise-driven dynamics.

These results confirm that while both thermostats eventually bring the system to the desired temperature range, only the Langevin thermostat exhibits physically plausible thermal fluctuations. The Berendsen method, despite its simplicity, fails to reproduce the statistical behavior of a canonical ensemble.

To investigate the numerical stability of the molecular dynamics integrator, we varied the timestep Δt in simulations of both two- and three-dimensional systems using the Langevin thermostat. The aim was to determine the largest timestep that yields stable dynamics while still correctly reproducing the target temperature. Table 1 summarizes which timesteps produced stable simulations and reports the corresponding average temperature. Simulations are marked unstable if temperature diverged or became physically unrealistic.

In the 2D simulations, timesteps up to $\Delta t = 0.02$ were found to be stable and accurately maintained the target temperature. Larger timesteps led to numerical instabilities. In 3D, the system was more sensitive to timestep size, with only $\Delta t = 0.01$ and below yielding stable trajectories.

3. Result and Analysis

Δt	Stability (2D)	Mean T (2D)	Stability (3D)	Mean T (3D)
0.002	Stable	0.1221	Stable	0.1917
0.010	Stable	0.1038	Stable	0.1074
0.020	Stable	0.1013	Unstable	—
0.050	Unstable	—	Unstable	—
0.100	Unstable	—	Unstable	—
0.150	Unstable	—	Unstable	—
0.200	Unstable	—	Unstable	—

Table 1: Timestep stability and mean temperature in 2D and 3D Langevin simulations.

These results are consistent with the corresponding temperature evolution plots (see figure 8b).

3. Result and Analysis

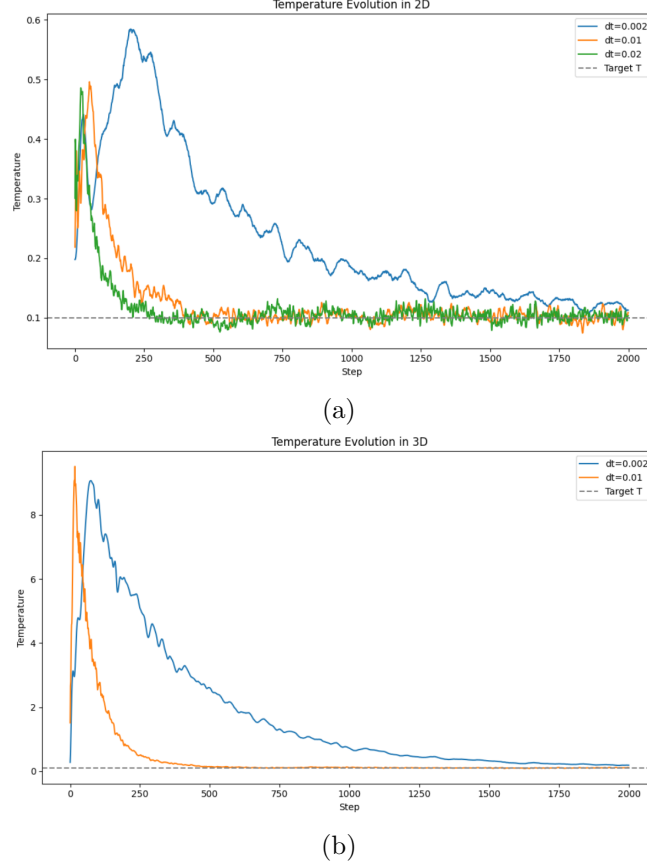


Figure 5: Temperature evolution in 2D and 3D systems with varying integration timestep Δt . In 2D (a), the system remains stable for all tested timesteps ($\Delta t = 0.002$, 0.01 , and 0.02), with higher timesteps leading to faster convergence toward the target temperature. In 3D (b), only $\Delta t = 0.002$ and 0.01 yield stable trajectories.

In 2D, increasing Δt from 0.002 to 0.02 improved convergence speed without compromising stability. In 3D, the simulation remained well-behaved at $\Delta t = 0.01$, but became unstable for $\Delta t = 0.02$ and beyond.

This analysis demonstrates that the Langevin thermostat enables larger stable timesteps, particularly in lower-dimensional systems, while still capturing correct thermal behavior. However, care must be taken when increasing Δt in higher dimensions, where instability can occur at smaller thresholds.

Next, the effect of varying the Berendsen thermostat parameter τ is investigated. The Berendsen thermostat regulates temperature by applying a velocity rescaling factor that depends on the relaxation time τ . This parameter controls how strongly the system is coupled to the heat bath: smaller values of τ correspond to faster temperature adjust-

3. Result and Analysis

ments, while larger values allow more gradual relaxation.

We performed a series of simulations with $\tau \in \{0.01, 0.02, 0.05, 0.1, 0.2, 1.0\}$, starting from the same condition. The figure 6 shows the time evolution of the system temperature for each case.

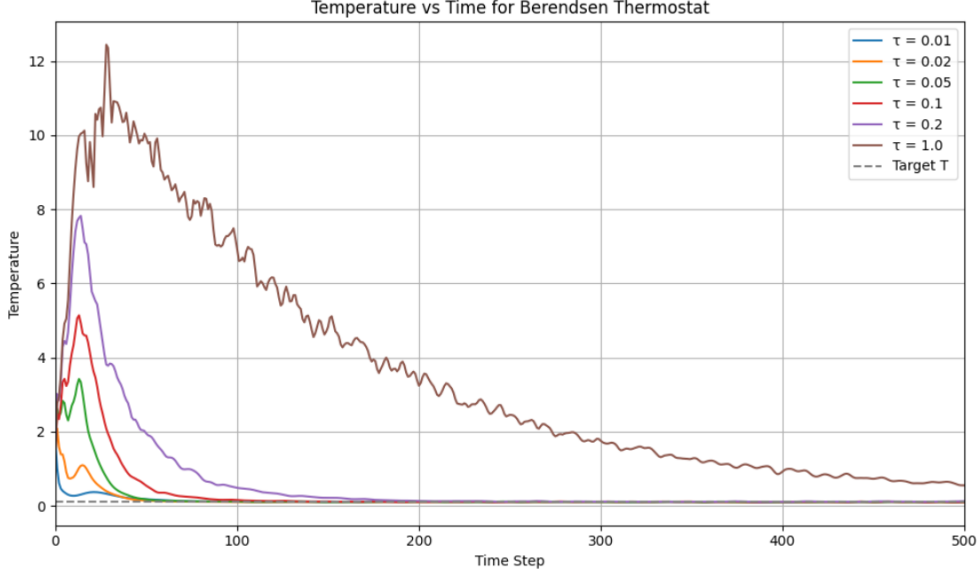


Figure 6: Temperature evolution for different values of the Berendsen thermostat parameter τ .

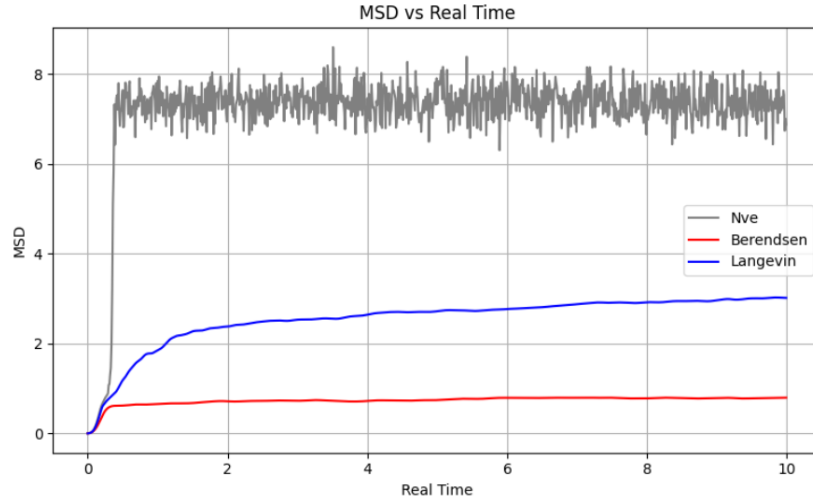
For small values ($\tau = 0.01$ to 0.1), the temperature is quickly driven toward the target value. The coupling to the thermostat is strong, leading to rapid damping of the initial high kinetic energy. As τ increases to 0.2 , the relaxation becomes slower, but the system still eventually approaches the target temperature. At $\tau = 1.0$, the thermostat acts very weakly, resulting in poor temperature control. The system remains far from equilibrium for most of the simulation duration. Thus, small τ values yield fast thermalization but artificially suppress fluctuations, while large τ values preserve dynamics better but may fail to maintain the desired temperature within a reasonable timescale.

To evaluate how different ensembles affect particle transport and temperature behavior, we measured the mean squared displacement (MSD) and extracted diffusion coefficients D for three cases: NVE, Berendsen, and Langevin. MSD was analyzed as a function of both simulation time (steps) and real physical time. Additionally, the temperature distributions were plotted to assess equilibrium and fluctuation properties for each system.

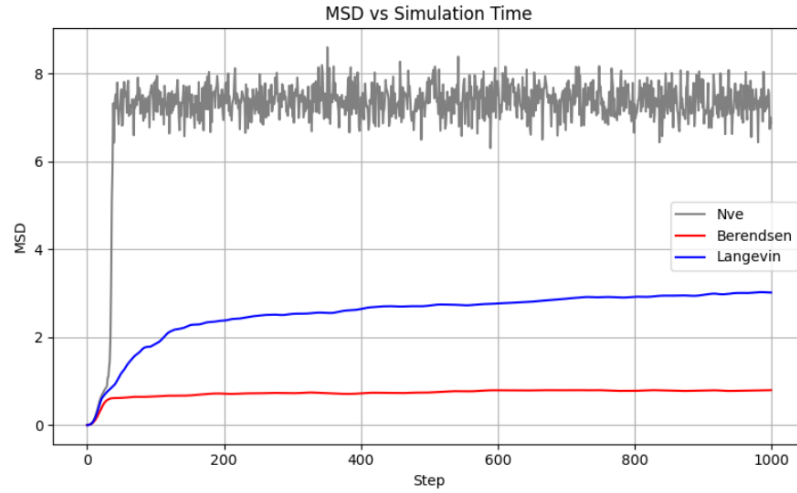
Figure 7 shows the MSD evolution over simulation steps and real time, respectively. In both plots, the Langevin system exhibits a higher diffusion rate than the Berendsen system, while the NVE system shows a rapid initial increase followed by large fluctuations

3. Result and Analysis

and noise.



(a)



(b)

Figure 7: Mean squared displacement (MSD) evolution as a function of real time (a) simulation steps (b) for three ensembles: NVE (no thermostat), Berendsen, and Langevin. The MSD increases most rapidly in the NVE system, but exhibits large fluctuations due to the absence of temperature regulation. The Langevin ensemble shows smooth diffusive behavior with moderate thermal noise, while the Berendsen ensemble displays the slowest and most damped particle motion due to strong velocity rescaling. In both representations, Langevin and Berendsen converge toward a steady diffusive regime, while the NVE dynamics remain unstable over time.

3. Result and Analysis

The diffusion coefficient was extracted by fitting the linear region of the MSD curves using

$$D = \frac{1}{2d} \frac{d}{dt} \text{MSD}(t)$$

with a fit window starting from step 700. The results are summarized in table 2.

Ensemble	Diffusion Coefficient D	Fit R^2
NVE	0.00249	0.0016
Berendsen	0.00855	0.9552
Langevin	0.00700	0.8185

Table 2: Estimated diffusion coefficients from MSD fits.

The Berendsen ensemble yielded the highest R^2 value, reflecting its low fluctuation and smooth dynamics. Langevin shows a physically realistic diffusion rate with moderate noise. The NVE case, as expected, was too noisy for a reliable diffusion estimate due to lack of thermal regulation.

Figure 8 shows the temperature distributions over the final 300 steps of each simulation.¹ The Berendsen thermostat yields a sharply peaked distribution centered near the target temperature, reflecting strong suppression of thermal fluctuations. The Langevin system exhibits a broader distribution, consistent with canonical ensemble behavior. The NVE ensemble shows a noisy, non-Gaussian distribution due to energy conservation and the absence of thermal damping.

¹NVE was unstable and omitted for clarity.

3. Result and Analysis

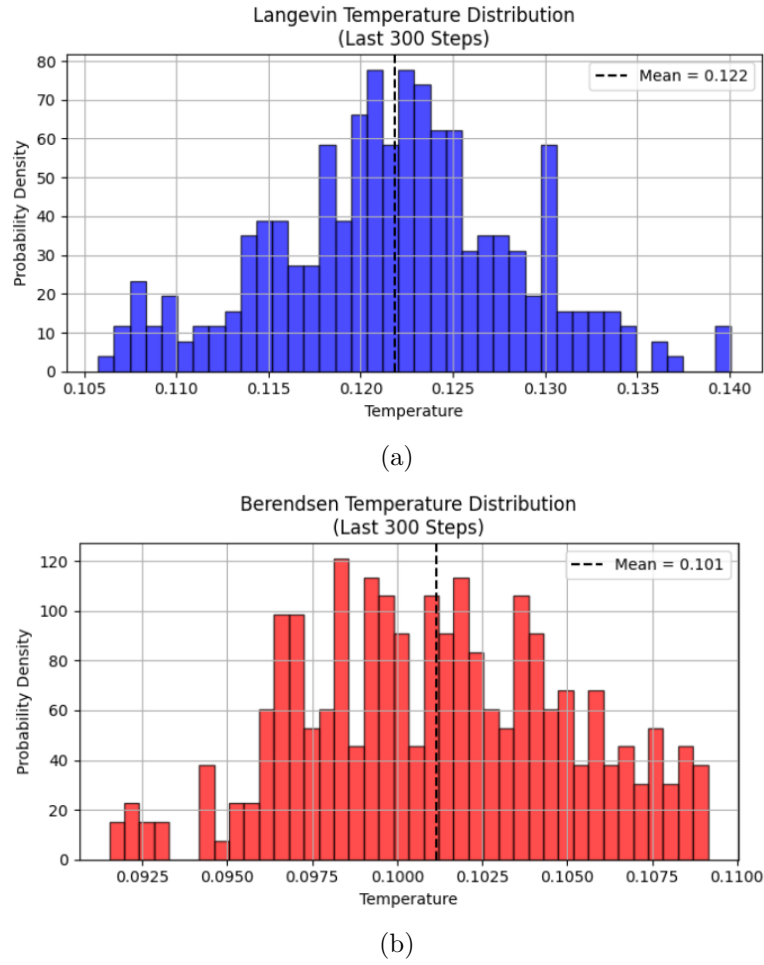


Figure 8: Temperature distribution for Berendsen and Langevin ensembles (last 300 steps).

This analysis confirms the expected behavior of each ensemble: NVE shows unreliable diffusion behavior and large temperature fluctuations due to the absence of a thermostat. Berendsen provides smooth, damped dynamics with suppressed fluctuations and an accurate diffusion fit, but lacks physical realism in fluctuation statistics. Langevin captures both realistic diffusive transport and thermal fluctuations, making it most appropriate for studies requiring canonical ensemble properties.

3. Result and Analysis

3.3. Temperature and Density Effect in 2D LJ simulation

In this section, we present our results and analysis of the MD simulation on the LJ particles. After running the simulation, we found that the trajectory can be found in the video link. The final snapshot is shown in Figure 9.

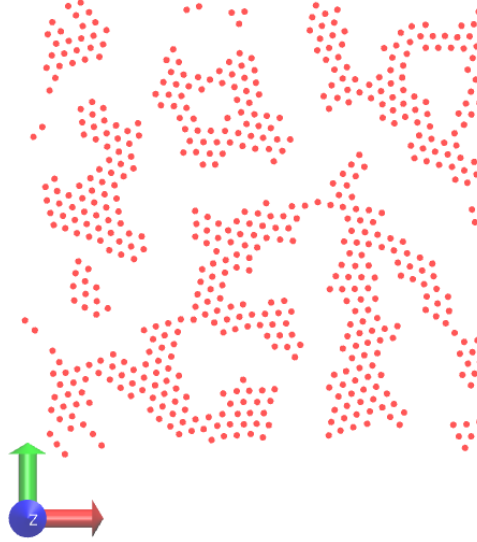


Figure 9: Final snapshot from LJ simulation. The detailed trajectory is given on GitHub.

Different density

In different densities, the particles in the box show different phases. Increasing density decreases the chance of particles moving in the box, thus increasing order in the box. Figure 10 shows a decrease in $g(r)$ with a decrease in periodicity. This indicates that if we decrease density, we will have less dense material. So, solid $\rightarrow$ softmaterial $\rightarrow$ liquid $\rightarrow$ gas.

3. Result and Analysis

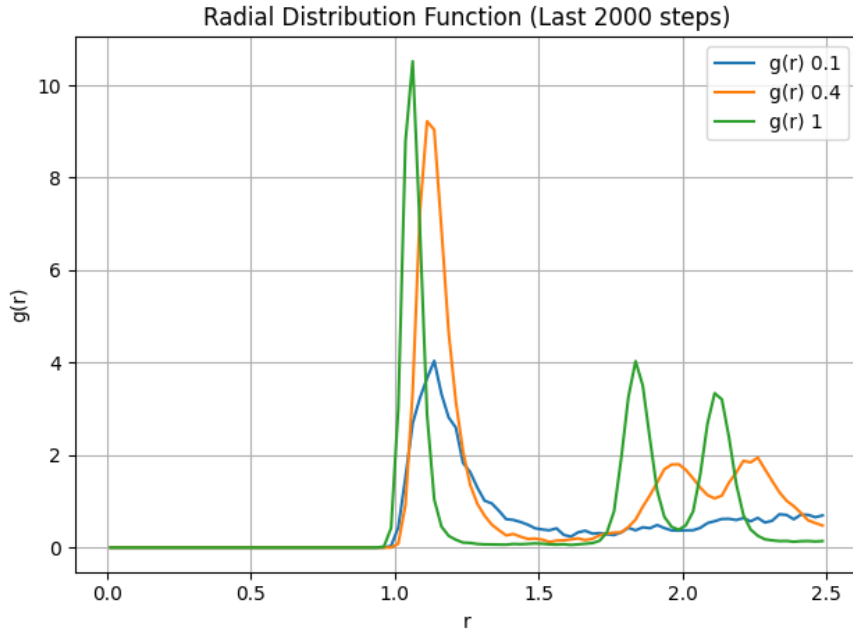
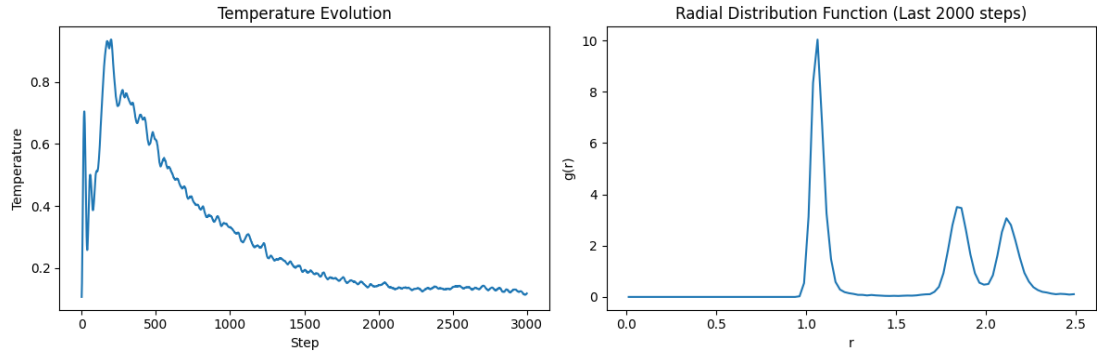


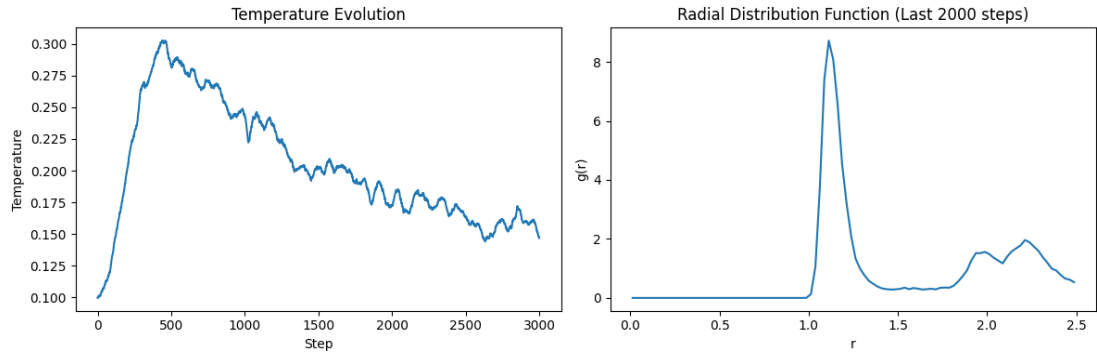
Figure 10: $g(r)$ vs r

If we take a look at the figure 11 We will see that increasing temperature leads to a break in the periodicity of the $g(r)$ function. If we increase the temperature to a ludicrous level, the temperature will fluctuate a lot, and the $g(r)$ will show fluctuation like the figure 11d, which indicates the gaseous phase of matter. The phase diagram of this simulation is left intentionally due to the fact that it will take a long amount of time.

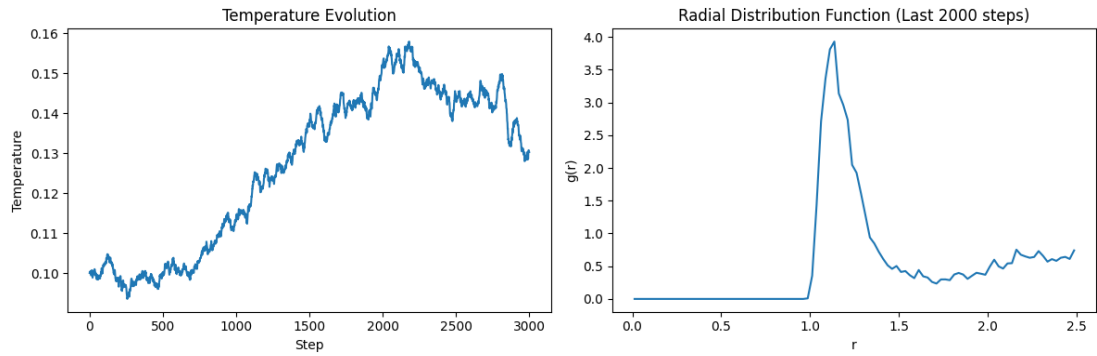
3. Result and Analysis



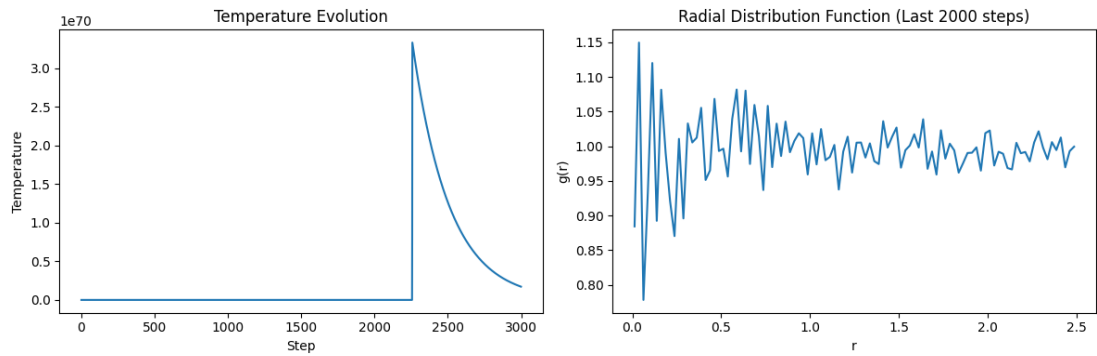
(a) Density =1.0, Temperature=0.1, Solid like periodicity



(b) Density=0.4, Temperature=0.1, Liquid-like dumped/noised periodicity.



(c) Density=0.1, Temperature=0.1, Liquid phase like particles.



(d) Density=0.1, Temperature=300, gas phase like particles.

Figure 11: MD simulation of LJ particles. The radial distribution indicates phases of material.

3. Result and Analysis

Temperature

From the MD simulation of LJ particles, we can also see the temperature effect. The temperature affects the dynamics of the simulation and thus the phase. If we increase the temperature, we can see the change in the phase behavior; again, increasing the temperature decreases the order, thus decreasing the periodicity, solid $\rightarrow$ softmatter $\rightarrow$ liquid $\rightarrow$ gas phase. Figure 8b shows the effect of temperature,

Even if we set our temperature at super high $T=300$, the simulation shows the particle moving really, really fast. This results in a fluctuating $g(r)$ vs r graph.

3.4. Monte Carlo Simulation

After running the simulation, we get the final snapshot of the 2D particle, which is shown in the figure 12.

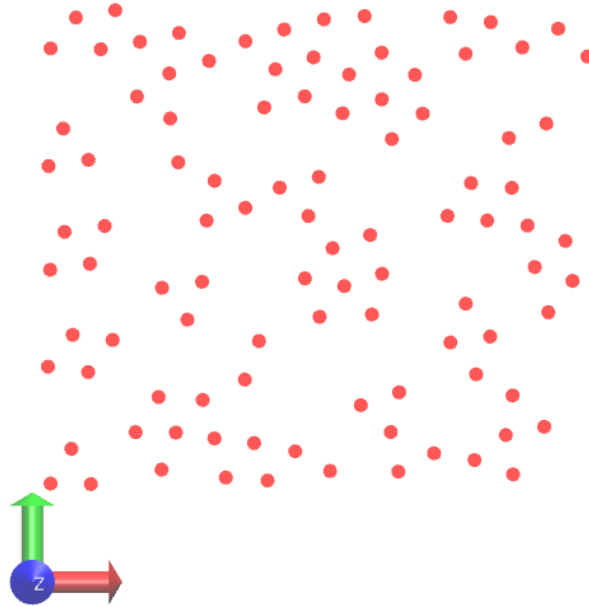


Figure 12: Final snapshot of the 2D particles

To get the optimized distance, we need to consider the following parameters,

- Maximum displacement radius
- Acceptance rate

3. Result and Analysis

- Final Energy value

If we vary the maximum displacement radius, we will see that there is an optimum distance where both final energies become negative and the acceptance rate stays above 0.30. From table 3, we can see that there are a couple of accepted `max_disp` values. But the largest `max_disp` value with acceptance rate threshold is 0.05.

Some details of the MC simulation results to judge the stability of the MC simulation are given in the appendix [4]. In all the MC simulations, we always try to maintain this acceptance rate. Figure 13 demonstrates the decrease in acceptance with increasing MC steps.

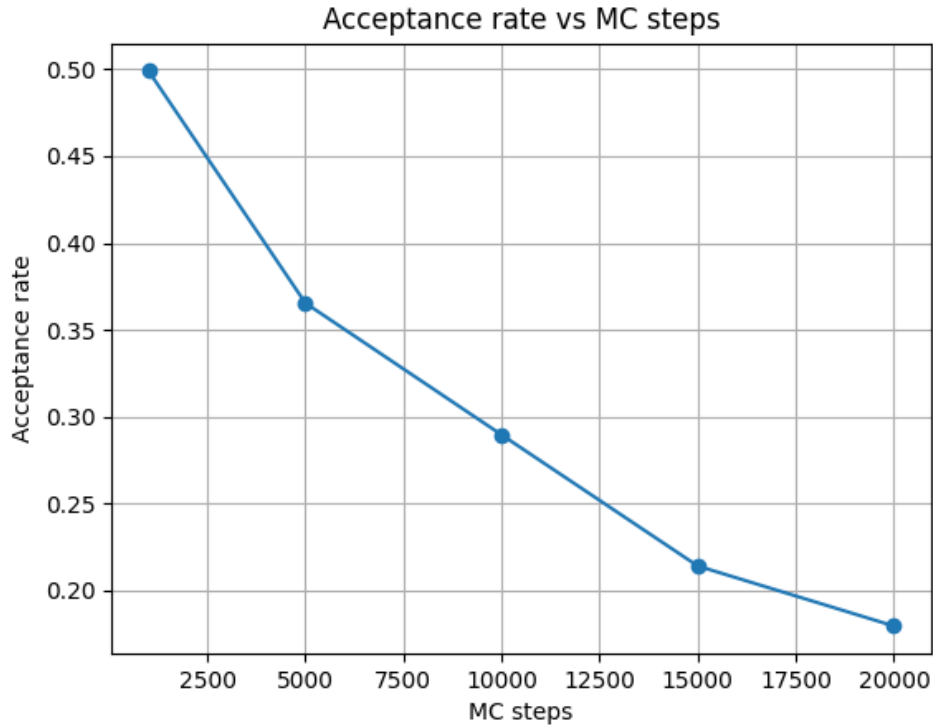


Figure 13: Acceptance vs MC steps

3.5. Effect of Density, Temperature, and Phase

Density effect

Like MD simulation, we vary the density from 0.1 to. And can detect phases of matter. Figure 15 shows us different phases of matter with respect to densities. The pattern is

3. Result and Analysis

the same as the MD simulation.

3.5.1. Temperature effect

We calculated the temperature by fixing the density. However, there was no significant changes in the radial distribution. This indicates that temperature is not affected in the MC simulation. As the origin of temperature comes from the kinetic energy/ dynamics of the simulation, it is easily understandable that there is no dynamics in the MC simulation. The figure is given in the Appendix 16.

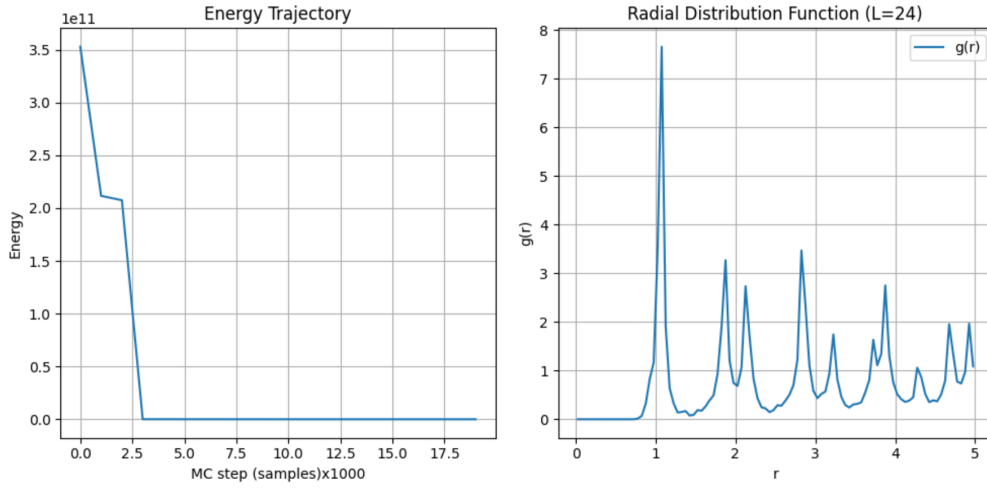
Now the big question is, can we replicate the results we get from MD?

Before answering this question, we need to keep in mind that MC and MD simulations are very different from each other. MD does have dynamics in it. Meanwhile, the MC simulation doesn't have dynamics. So, the only thing we can compare is their final snapshot/configuration.

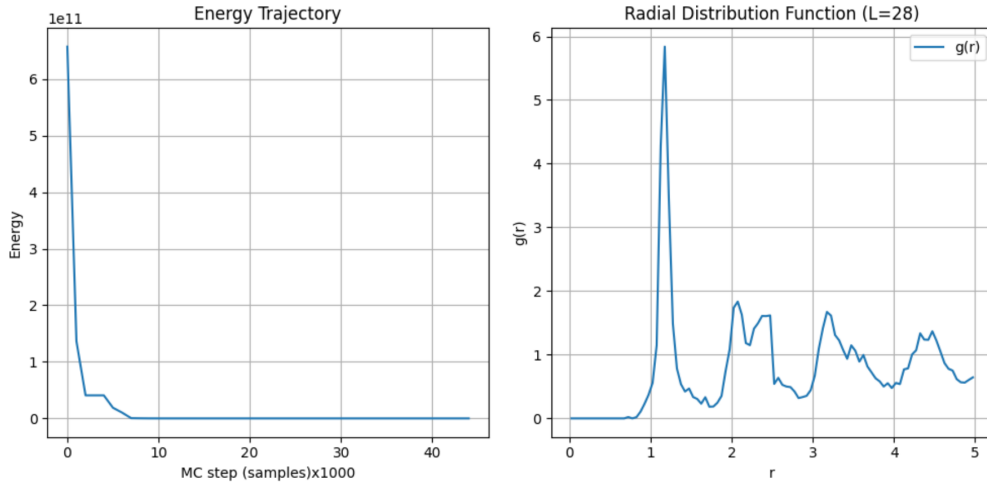
From the ergodicity, we can say that if we take a trajectory of a simulation long enough to cover the phase space, and the MC simulation accepts conformations enough to cover the phase space, they should be comparable. So, how to confirm that our MC simulation is replicating the results of MD? The first thing we can check is if there is any local minimum energy point where the MC simulation might be trapped. Secondly, we can vary the temperature and density and check if any abnormality comes up in the final confirmation, energy, or acceptance rate.

In our experimentation, after varying the density and temperature, we didn't see such anomalies. The figure shows the expected phases of matter across the parameter space. In this case, the Monte Carlo (MC) simulation successfully replicates the results of the Molecular Dynamics (MD) simulation.

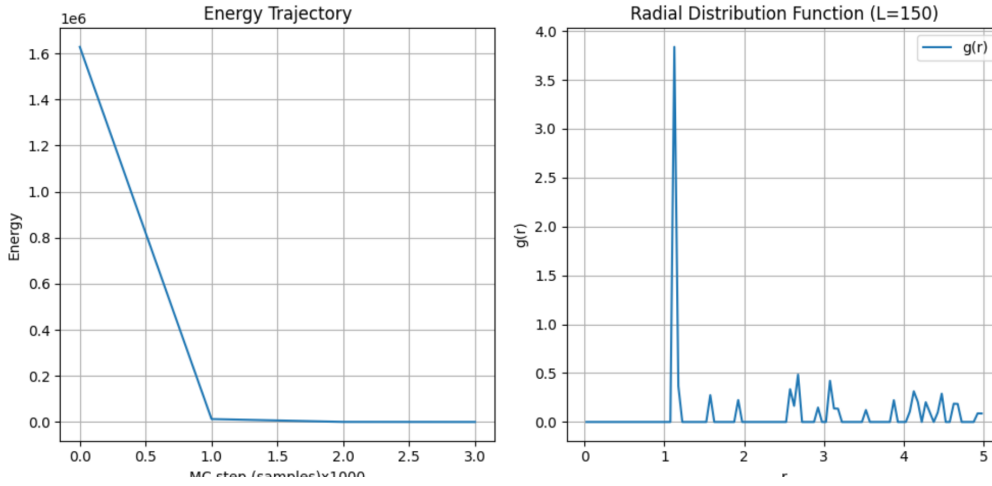
3. Result and Analysis



(a) Density=1, sharp periodic spikes indicate solid phase of matter.



(b) Density=0.734, indicate soft matter/liquid.

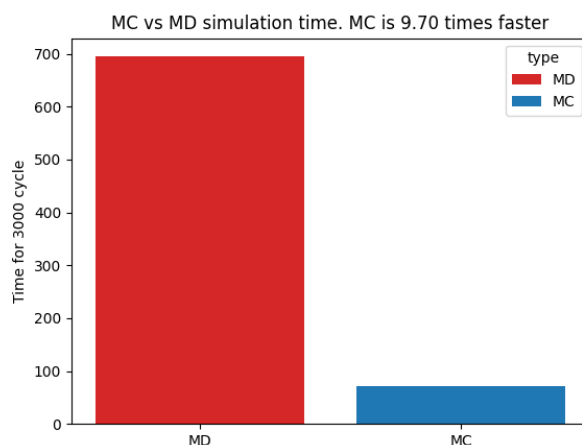


(c) Density=0.26, indicates gaseous state.

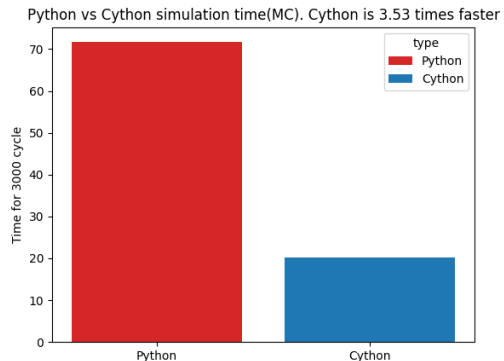
Figure 14: Phases from MC simulation.

3.6. Computational efficiency and performance Optimization

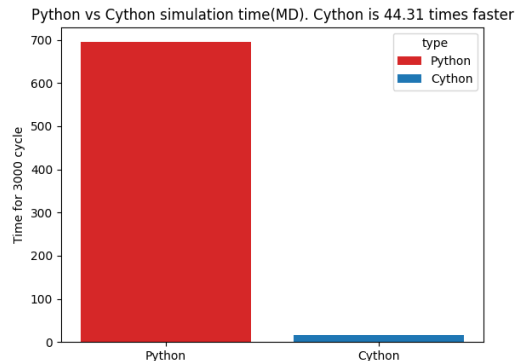
When doing MC simulation, we have the advantage of getting the equilibrated conformation at a really fast rate. Even though the concept of both simulations is not the same, for generating a stable conformation, the MC simulation is lightning fast. **For example, here, for 576 LJ particles, a run of 3000 cycles takes around 694.98 seconds for the MD simulation, while the same cycles only need 71.67 seconds in the MC simulation. Almost 10 times.**



(a) MC vs MD time comparison



(b) Cython optimization (MC)



(c) Cython optimization (MD)

Figure 15: Time comparison in (a)MD vs MC, (b) Cython vs Python(MC), (c) Cython vs Python (MD)

4. Conclusion

3.7. Cython Optimization

As Python was taking too much time to calculate, we tried to optimize it using Cython [4]. **Figure 15b and Figure 15c show that the MC and MD simulation is optimized by Cython about 3.53 and 44.31 times faster, respectively.**

Code and Data Availability

All the calculations were performed using Python and Cython. The code and images are provided in the supplementary material, available at the following GitHub repository: <https://github.com/shafayetrahat/Thermostats-and-MC-simulation>.

4. Conclusion

These exercises highlighted how complementary MD and MC are for computational statistical mechanics. While MD tracks real dynamics, MC efficiently samples phase space, making them powerful tools for understanding many-body systems.

References

- [1] Alex Bunker. Molecular modelling. *Lecture at University of Helsinki (KEM 342)*, 2025.
- [2] Alex Bunker. Programming projects in molecular modelling. *Lecture at University of Helsinki (KEM 381)*, 2025.
- [3] ETH Zurich. Computational statistical physics – exercise 8, 2025. Accessed: 2025-03-25.
- [4] Stefan Behnel, Robert Bradshaw, Craig Citro, Lisandro Dalcin, Dag Sverre Seljebotn, and Kurt Smith. Cython: The best of both worlds. *Computing in Science & Engineering*, 13(2):31–39, 2011.

5. Appendix

5. Appendix

A. Graphs of MC simulation. Varied temperature

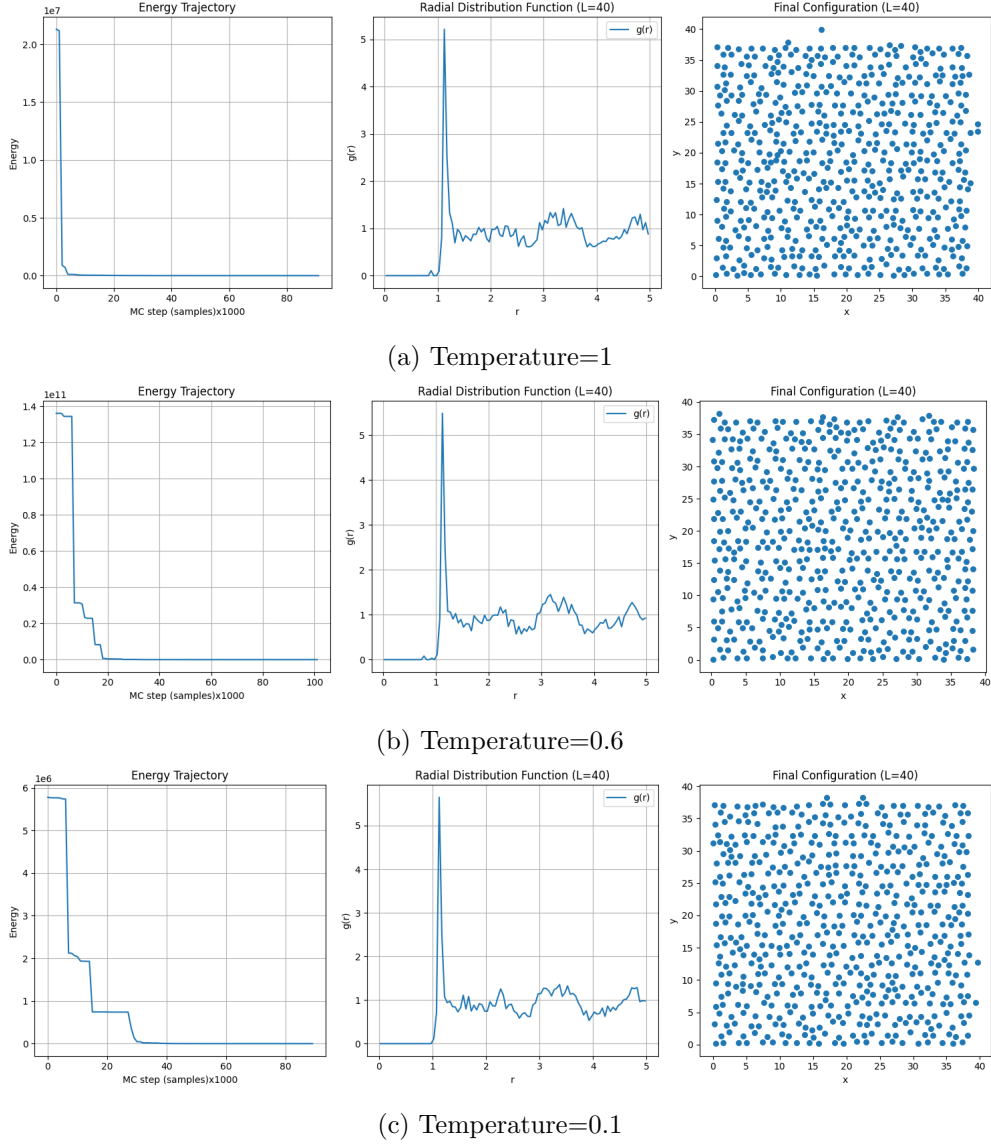


Figure 16: Phases from MC simulation. All graphs indicates no dynamics can be found from the MC simulation.

B. Tables

max_disp	acceptance_rate	min_energy	time
0.0041753189365604	0.48828571428571427	8367.60	8.106
0.005736152510448682	0.4937142857142857	5082.49	8.185
0.007880462815669913	0.5017142857142857	49232.38	8.484
0.01082636733874054	0.49214285714285716	5322.68	7.634
0.014873521072935117	0.49357142857142855	-86.97	8.050
0.020433597178569417	0.4848571428571429	-104.30	7.524
0.028072162039411756	0.4427142857142857	-124.25	7.435
0.03856620421163472	0.38157142857142856	-137.51	7.497
0.05298316906283707	0.3057142857142857	-142.06	7.481
0.07278953843983145	0.24785714285714286	-149.78	7.680

Table 3: MC simulation data.

C. List of MD simulations core functions and Description

`compute_lj_force(DTYPE_t r, DTYPE_t sigma, DTYPE_t epsilon, DTYPE_t rcutoff)`

- **Arguments:** Interparticle distance (`r`), LJ parameters (`sigma`, `epsilon`), cutoff distance (`rcutoff`)
- **Returns:** LJ force magnitude
- **Description:** Computes 12-6 Lennard-Jones force with cutoff

`compute_lj_potential(DTYPE_t r, DTYPE_t sigma, DTYPE_t epsilon, DTYPE_t rcutoff)`

- **Arguments:** Same as `compute_lj_force`
- **Returns:** LJ potential energy (shifted)
- **Description:** Calculates shifted LJ potential

`create_lattice(int n_particles, DTYPE_t box_size, int dimensions)`

- **Arguments:** Particle count, box size, system dimensions (2D/3D)
- **Returns:** Initial particle positions
- **Description:** Creates triangular lattice with random perturbations

`initialize_velocities(int n_particles, int dimensions, DTYPE_t target_temp)`

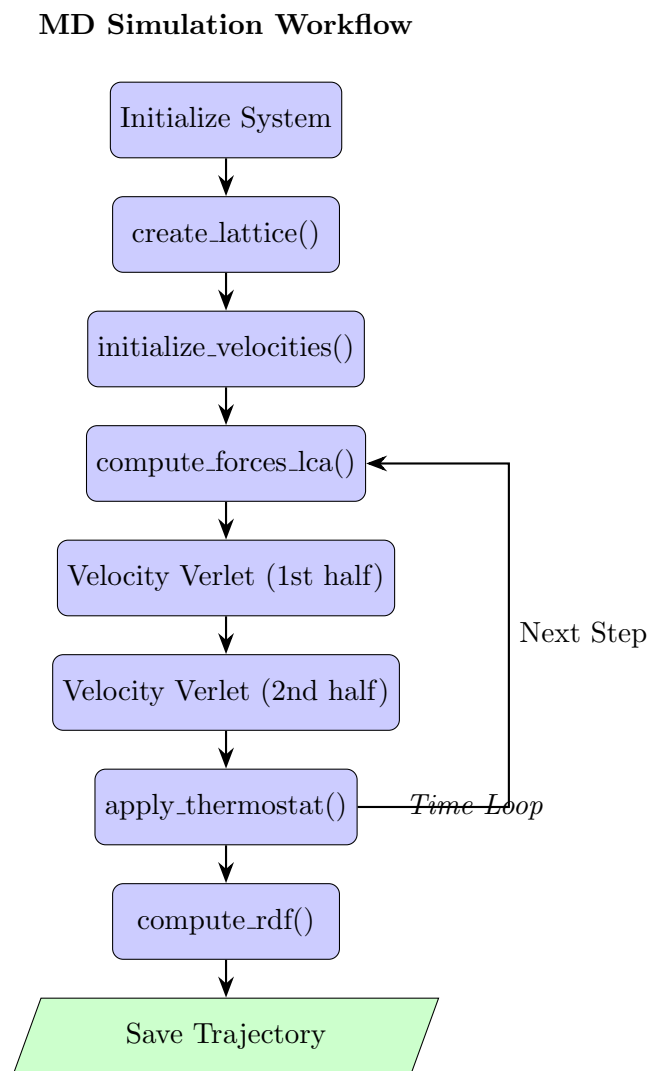


Figure 17: Flow diagram of MD simulation code

C. List of MD simulations core functions and Description

- **Arguments:** Particle count, dimensions, target temperature
- **Returns:** Initial velocity array
- **Description:** Generates Maxwell-Boltzmann distributed velocities

Thermostats

`apply_berendsen_thermostat(velocities, target_temp, current_temp, dt, tau)`

- **Arguments:** Velocity array, target/current temperatures, timestep, relaxation time
- **Returns:** Scaled velocities
- **Description:** Applies Berendsen velocity rescaling

`apply_langevin_thermostat(velocities, gamma, dt, target_temp)`

- **Arguments:** Velocity array, friction coefficient, timestep, target temperature
- **Returns:** Thermostated velocities
- **Description:** Implements Langevin dynamics with random kicks

Force Calculation

`compute_forces_lca(positions, box_size, rcutoff, sigma, epsilon, use_pbc)`

- **Arguments:** Position array, box size, cutoff, LJ parameters, PBC flag
- **Returns:** (forces, potential_energy)
- **Description:** Computes forces using minimum image convention

Analysis Functions

`compute_rdf(positions, box_size, rcutoff, n_bins=RDF_BINS)`

- **Arguments:** Position array, box size, cutoff, bin count
- **Returns:** (r_values, g_r)
- **Description:** Calculates radial distribution function

D. List of MC simulations core functions and Description

Main Simulation

`run_simulation(params, trajectory_save=False, ...)`

- **Arguments:** Parameter dictionary and control flags
- **Returns:** Dictionary with results
- **Description:** Main MD driver with velocity Verlet integration

`run_simulation_py(...)`

- **Arguments:** Python-friendly parameter interface
- **Returns:** Same as `run_simulation`
- **Description:** Python wrapper for the Cython implementation

D. List of MC simulations core functions and Description

`initialize_lattice(int N, DTYPE_t L, DTYPE_t rho, DTYPE_t rc)`

- **Arguments:**
 - N: Number of particles
 - L: Box length
 - rho: Number density
 - rc: Cutoff distance
- **Returns:** Tuple of (positions, total_energy)
- **Description:** Initializes particles on a 2D triangular lattice with random fill

`periodic_distance(np.ndarray[DTYPE_t] r1, np.ndarray[DTYPE_t] r2, DTYPE_t L)`

- **Arguments:**
 - r1, r2: Position vectors
 - L: Box length
- **Returns:** Minimum image distance

Monte Carlo Simulation Workflow

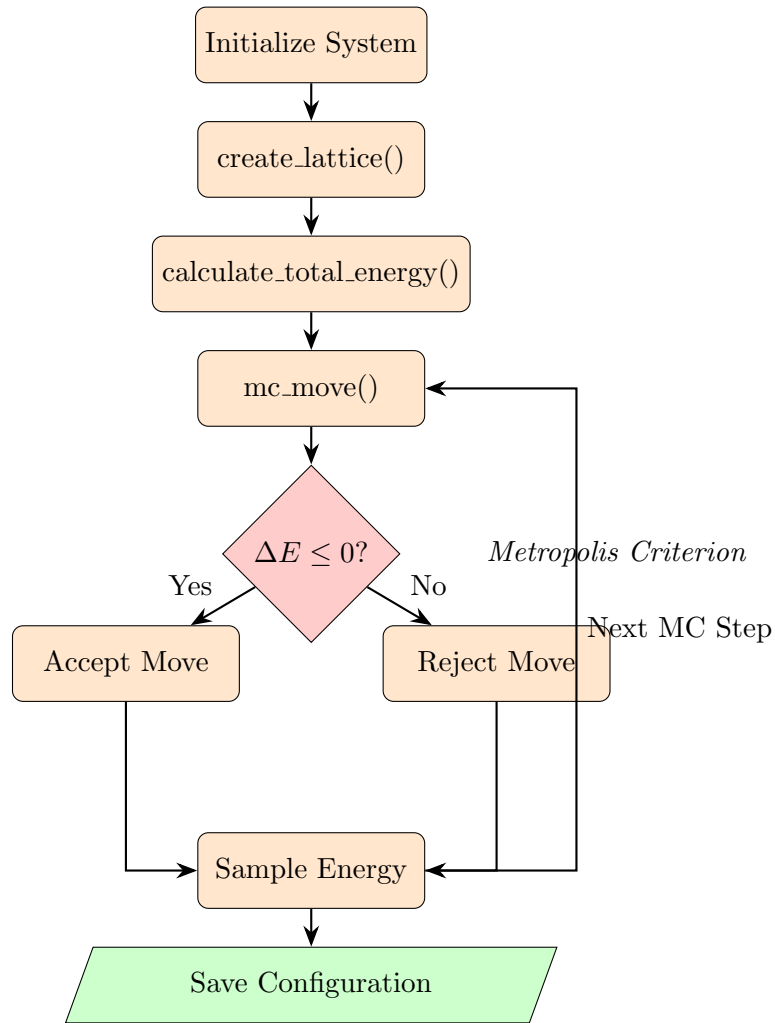


Figure 18: Flow diagram of MC simulation

D. List of MC simulations core functions and Description

- **Description:** Computes shortest distance between particles under periodic boundary conditions

`lj_potential(DTYPE_t r, DTYPE_t rc)`

- **Arguments:**
 - `r`: Distance between particles
 - `rc`: Cutoff distance
- **Returns:** LJ potential energy
- **Description:** Computes 12-6 Lennard-Jones potential with cutoff

`mc_move(positions, N, L, rc, max_disp, beta)`

- **Arguments:**
 - `positions`: Current particle positions
 - `N`: Number of particles
 - `L`: Box length
 - `rc`: Cutoff distance
 - `max_disp`: Maximum displacement
 - `beta`: $1/(k_B T)$
- **Returns:** Tuple of (`new_positions`, `deltaE`, `acceptance_flag`)
- **Description:** Performs single Monte Carlo move with Metropolis criterion

`run_simulation(N, L, T, rc, max_disp, steps, equil_steps, sample_freq)`

- **Arguments:**
 - `N`: Number of particles
 - `L`: Box length
 - `T`: Temperature
 - `rc`: Cutoff distance
 - `max_disp`: Maximum displacement
 - `steps`: Production steps

D. List of MC simulations core functions and Description

- `equil_steps`: Equilibration steps
- `sample_freq`: Energy sampling frequency
- **Returns:** Tuple of (final_positions, energy_array, acceptance_rate)
- **Description:** Main simulation driver performing equilibration and production MC runs